PriVault

"Your files. Your vault. Your privacy."

Comprehensive Project Report

Generated: March 26, 2026

Version 1.0.0+1 | Flutter / Dart | Android

Firebase | Cloudinary | XChaCha20-Poly1305

Table of Contents

Section 1

Project Overview

PriVault is a privacy-first encrypted cloud storage mobile application built with Flutter. It enables users to securely store, manage, and share files with client-side encryption using the XChaCha20-Poly1305 authenticated encryption algorithm.

Property	Value
App Name	PriVault
Tagline	Your files. Your vault. Your privacy.
Purpose	Secure encrypted file storage mobile app
Platform	Android (Flutter)
Version	1.0.0+1
Min SDK	Dart ^3.6.0
Package ID	pri_vault

PriVault employs a zero-knowledge architecture where encryption keys are derived on the client device and never transmitted to the server. All files are encrypted before upload, and the server only ever stores ciphertext.

Section 2

Tech Stack

Layer	Technology	Details
Frontend	Flutter (Dart)	Cross-platform UI framework, Material 3 dark theme
State Mgmt	Riverpod	flutter_riverpod ^2.6.1, StateNotifier pattern
Navigation	GoRouter	go_router ^14.8.1, declarative routing with shell routes
Authentication	Firebase Auth	Email/password + Google Sign-In
Database	Cloud Firestore	NoSQL document database, real-time streams
File Storage	Cloudinary	Cloud name: dqed2mebk, unsigned uploads
Local Storage	Hive + SecureStorage	Onboarding flags, key caching
Encryption	XChaCha20-Poly1305	Per-file keys, Argon2id master key derivation
Key Derivation	Argon2id + HKDF	64MB memory, 3 iterations, 4 parallelism
Key Exchange	X25519	ECDH for file sharing between users
OCR	Google ML Kit	Text recognition for CamScanner feature
Typography	Google Fonts (Poppins)	Material 3 type scale

Section 3

Project Structure

The lib/ directory follows a feature-first architecture with shared core utilities:

```
lib/
+-- main.dart                  -- App entry point (Firebase, Hive, Riverpod)
+-- firebase_options.dart      -- FlutterFire generated config
+-- core/
|   +-- api/api_client.dart         -- Firebase providers (Auth, Firestore)
|   +-- constants.dart              -- Crypto, storage, app constants
|   +-- utils.dart                  -- Formatting, validation helpers
|   +-- encryption/
|   |   +-- encryption_service.dart -- XChaCha20-Poly1305 encrypt/decrypt
|   |   +-- crypto_utils.dart       -- Nonce, key gen, base64, wipe
|   |   +-- key_derivation.dart     -- Argon2id + HKDF sub-keys
|   +-- router/app_router.dart      -- GoRouter config & route constants
|   +-- theme/app_theme.dart        -- Dark Material 3 theme & colors
|   +-- validators/validators.dart  -- Password & email validators
+-- models/
|   +-- user_model.dart       -- User profile model
|   +-- file_model.dart       -- File metadata model
|   +-- file_metadata.dart    -- Freezed file metadata
|   +-- folder_model.dart     -- Folder model
|   +-- folder.dart           -- Freezed folder
|   +-- share_model.dart      -- Share link/user model
|   +-- share_models.dart     -- Share, ShareRecipient, SharedFile
|   +-- profile.dart          -- Freezed profile model
|   +-- file_version.dart     -- File version tracking
+-- repositories/
|   +-- storage_repository.dart  -- File CRUD, upload, download, decrypt
|   +-- share_repository.dart    -- Sharing, teams, permissions
|   +-- profile_repository.dart  -- Profile read/update
+-- services/
|   +-- auth_service.dart          -- Firebase Auth + profile creation
|   +-- cloudinary_service.dart    -- Cloudinary upload/delete
|   +-- vault_service.dart         -- Secure key storage + Firestore sync
|   +-- storage_service.dart       -- Storage quota management
|   +-- audit_service.dart         -- Audit logging to Firestore
|   +-- notification_service.dart  -- Push notification events
+-- features/
|   +-- auth/         -- Login, Signup, Forgot Password screens + providers
|   +-- dashboard/    -- Home screen with storage stats, recent files
|   +-- files/        -- File management, upload, download, search
|   +-- secure_vault/ -- Encrypted personal vault
|   +-- sharing/      -- File sharing with permissions, teams
|   +-- company/      -- Company accounts and teams
|   +-- papers_wallet/-- Digital cards storage (ID, bank cards)
|   +-- audit/        -- Audit logs screen
|   +-- camscanner/   -- Document OCR scanning
```

```
|   +-- comments/     -- File comments widget
|   +-- notifications/-- Notifications screen
|   +-- onboarding/   -- Splash + onboarding screens
|   +-- settings/     -- Profile & app settings
|   +-- setup/        -- Username setup, plan selection, edit profile
|   +-- trash/        -- Deleted files (30-day retention)
+-- ui/
    +-- widgets/bottom_nav.dart -- Navigation shell with bottom bar
```

Section 4

Firebase / Firestore Database Schema

All Firestore collections and their field structures, derived from actual source code:

COLLECTION: users/{uid}

Field	Type	Description
uid	String	Firebase Auth UID
email	String	User email address
displayName	String?	Username (set after signup)
photoURL	String?	Cloudinary profile photo URL
phoneNumber	String?	Phone with country code
accountType	String	'regular' or 'company'
plan	String	'free', 'premium', 'professional'
storageUsedBytes	Integer	Current storage used in bytes
storageMaxBytes	Integer	Max storage (3GB=3221225472)
salt	String	Base64 Argon2id salt
publicKey	String	Base64 X25519 public key
masterKeySeed	String	Base64 master key (Firestore backup)
createdAt	Timestamp	Account creation date
lastLoginAt	Timestamp	Last login timestamp

COLLECTION: users/{uid}/files/{fileId}

Field	Type	Description
name	String	Encrypted filename (Base64)
encryptedName	String	Encrypted original filename (Base64)
mimeType	String	Original MIME type (e.g. image/jpeg)
sizeBytes	Integer	Original file size in bytes
cloudinaryUrl	String	Cloudinary secure URL of ciphertext
fileKeyEncrypted	String	Per-file key encrypted with master key (Base64)
folderId	String?	Parent folder ID (null = root)
isDeleted	Boolean	Soft-delete flag (trash)
deletedAt	Timestamp?	When file was moved to trash
isFavorite	Boolean	User favorite flag
isVaultFile	Boolean	true if stored in secure vault
version	Integer	File version number
encryptionNonce	String	Base64 XChaCha20 nonce (24 bytes)
encryptionSalt	String	Base64 encryption salt
encryptionMac	String	Base64 Poly1305 MAC
encryptionAlgo	String	'xchacha20-poly1305'

createdAt	Timestamp	Upload timestamp
updatedAt	Timestamp	Last modified timestamp

COLLECTION: folders/{folderId}

Field	Type	Description
userId	String	Owner UID
name	String	Folder name
parentId	String?	Parent folder ID (null = root)
color	String?	Custom folder color
isShared	Boolean	Shared flag
createdAt	Timestamp	Creation timestamp
updatedAt	Timestamp	Last modified timestamp

COLLECTION: shares/{shareId}

Field	Type	Description
owner_id	String	Sharer's UID
owner_email	String	Sharer's email
file_id	String?	Shared file ID
file_name	String	Encrypted file name
folder_id	String?	Shared folder ID
type	String	'link', 'user', or 'team'
target_id	String?	Recipient UID or team ID
shared_with_email	String?	Recipient email (user shares)
team_id	String?	Team ID (team shares)
encrypted_key	String	Re-encrypted file key for recipient
recipient_encrypted_key	String?	Recipient-specific encrypted key
sender_public_key	String?	Sender X25519 public key (Base64)
password_hash	String?	Optional share password hash
permission	String	'view' or 'download'
max_downloads	Integer	Download limit (0 = unlimited)
download_count	Integer	Current download count
expires_at	String?	ISO8601 expiration date
is_revoked	Boolean	Whether share has been revoked
revoked_at	Timestamp?	Revocation timestamp
created_at	Timestamp	Share creation date

COLLECTION: sharedAccess/{fileId}_{userId}

Field	Type	Description
file_id	String	Shared file ID
owner_id	String	File owner UID
user_id	String	Recipient UID

permission	String	'view' or 'download'
expires_at	String?	ISO8601 expiration
team_id	String?	Team ID if team share
created_at	Timestamp	Access grant timestamp

COLLECTION: fileComments/{fileId}/comments/{commentId}

Field	Type	Description
userId	String	Commenter UID
displayName	String	Commenter display name
avatarUrl	String?	Commenter avatar URL
text	String	Comment text (max 750 chars)
createdAt	Timestamp	Comment timestamp

COLLECTION: auditLogs/{logId}

Field	Type	Description
userId	String	Actor UID
email	String	Actor email
action	String	Action string (e.g. 'auth.login')
timestamp	Timestamp	When the action occurred
deviceType	String	Device platform (Android, iOS, Web)
metadata	Map?	Optional extra data about the action

COLLECTION: teams/{teamId}

Field	Type	Description
name	String	Team name
owner_id	String	Team creator UID
created_at	Timestamp	Creation timestamp

COLLECTION: teamMembers/{docId}

Field	Type	Description
team_id	String	Parent team ID
user_id	String	Member UID
added_by	String	UID of the user who added this member
created_at	Timestamp	When member was added

COLLECTION: notifications/{notifId}

Field	Type	Description
recipientId	String	Notification recipient UID

type	String	Event type (e.g. 'file_shared')
fileId	String	Related file ID
fileName	String	Related file name
actor.id	String	Actor UID
actor.name	String	Actor display name
timestamp	Timestamp	Notification timestamp
isRead	Boolean	Read status

Section 5

Features List

Complete feature inventory with implementation status:

[OK] User registration with email/password
[OK] Google Sign-In
[OK] Forgot password (Firebase reset email flow)
[OK] Change password (with re-authentication)
[OK] Username and profile photo setup
[OK] Plan selection (Free / Premium / Professional)
[OK] File upload with client-side XChaCha20-Poly1305 encryption
[OK] File download with decryption
[OK] File categorization (Images, Documents, Videos, Music)
[OK] Folder management (create, rename, delete, navigate)
[OK] Encrypted file search
[OK] File sharing with permissions (view/download) and expiry
[OK] Public link sharing with re-encrypted keys
[OK] User-to-user sharing via X25519 key exchange
[OK] Team sharing
[OK] Revoke sharing
[OK] Secure vault (encrypted personal vault)
[OK] Papers wallet (ID cards, bank cards)
[OK] Document scanner (CamScanner with Google ML Kit OCR)
[OK] File comments (max 750 chars, Firestore-backed)
[OK] Trash bin (30-day auto-delete concept)
[OK] Permanent deletion (Cloudinary + Firestore cleanup)
[OK] Dashboard with real storage statistics
[OK] Storage usage tracking (real-time quota enforcement)
[OK] Audit logs (auth, file, share events)
[OK] Company accounts
[OK] Team management (create, add members)
[OK] Onboarding screens (4 pages)
[OK] Dark theme (Material 3, deep dark design)
[OK] Notifications (in-app, Firestore-backed)
[OK] Edit profile (name, photo, password change)
[WARN] Premium/Pro plan payments (UI only, no real IAP)
[WARN] Cloudinary file deletion (requires signed backend)
[--] Biometric/PIN authentication (removed by design)

[--] Company vault (not implemented)

Section 6

Encryption Architecture

PriVault implements a zero-knowledge encryption architecture where all cryptographic operations happen on the client device. The server never sees plaintext data or encryption keys.

1. Master Key Generation

On signup, a 16-byte random salt is generated. The user's password and salt are fed into Argon2id (64MB memory, 3 iterations, 4 parallelism) to derive a 256-bit master key. The salt is stored in Firestore (users/{uid}.salt). The master key itself is stored locally in FlutterSecureStorage and backed up as masterKeySeed in Firestore.

2. Per-File Key Generation

Each file upload generates a fresh 256-bit random key using a cryptographically secure random number generator (dart:math Random.secure()).

3. File Encryption Before Upload

The file plaintext is encrypted using XChaCha20-Poly1305 with the per-file key and a fresh 24-byte nonce. The output format is: nonce (24B) || ciphertext || MAC (16B). This packed ciphertext is uploaded to Cloudinary.

4. File Key Encryption with Master Key

The per-file key is encrypted with the master key using the same XChaCha20-Poly1305 algorithm, producing fileKeyEncrypted which is stored in Firestore.

5. Master Key Seed in Firestore

The master key (as Base64) is stored in users/{uid}.masterKeySeed for cross-device access. On login, the key is also re-derived from the password + stored salt.

6. File Name Encryption

Original filenames are encrypted with the master key using XChaCha20-Poly1305 and stored as Base64 in the encryptedName field. This prevents metadata leakage.

7. Decryption Flow

1) Retrieve master key from VaultService (local cache or Firestore fallback).
2) Decrypt fileKeyEncrypted using master key to recover the per-file key.
3) Download ciphertext from Cloudinary URL.
4) Decrypt ciphertext using the per-file key (extracting nonce from first 24 bytes).
5) Optionally verify nonce matches the stored encryptionNonce for integrity.

8. Sharing Encryption (X25519)

For user-to-user sharing: Sender derives a shared secret via X25519 ECDH using their private key + recipient's public key. The file key is re-encrypted with this shared secret, allowing the recipient to decrypt without knowing the owner's master key.

Section 7

App Screens

Complete list of all application screens and their purposes:

Screen	Route	Description
Splash Screen	/	Animated splash with auto-navigation based on auth state
Onboarding (4 pages)	/onboarding	First-time user introduction to app features
Login Screen	/login	Email/password + Google Sign-In authentication
Sign Up Screen	/signup	New account registration with password validation
Forgot Password	/forgot-password	Firebase password reset email flow
Username Setup	/username-setup	Post-signup username configuration
Plan Selection	/plan	Free/Premium/Professional plan chooser
Dashboard	/home	Home screen with storage stats, recent files, categories
My Files	/files	File browser with folders, upload, search
File Viewer	(modal)	Image/PDF/video viewer with comments and actions
Search	(in-files)	Encrypted filename search with live results
Sharing	/sharing	Manage shared files, create/revoke shares
Shared With Me	(tab)	Files shared by other users
Create Share	(sheet)	Share dialog with permissions and expiry
Secure Vault	/secure-vault	Personal encrypted vault (no sharing)
Papers Wallet	/papers-wallet	Digital ID and bank card storage
CamScanner	/camscanner	Document scanning with OCR text extraction
Settings/Profile	/settings	Profile info, theme, about, logout
Edit Profile	/edit-profile	Edit name, photo, change password
Company	/company	Company account management and teams
Audit Logs	/audit-logs	View authentication and file action history
Trash	/trash	Deleted files with restore/permanent delete
Notifications	/notifications	In-app notification center

Section 8

Cloudinary Integration

Parameter	Value
Cloud Name	dqed2mebk
Upload Preset	privault_uploads
API URL	https://api.cloudinary.com/v1_1/dqed2mebk/auto/upload
Upload Method	HTTP POST (unsigned, multipart)
Content Type	application/octet-stream (all files encrypted)

File Path Structure

Category	Cloudinary Path
User Files	users/{uid}/files/{fileId}
Profile Photos	users/{uid}/avatar_{timestamp}
Vault Files	users/{uid}/files/{fileId} (isVaultFile=true)
Company Files	companies/{companyId}/files/{fileId}

Upload Flow

1. File is read as bytes from the device.
2. Plaintext is encrypted with per-file key using XChaCha20-Poly1305.
3. Packed ciphertext (nonce || ciphertext || MAC) is uploaded to Cloudinary.
4. Cloudinary returns a secure_url which is stored in Firestore.
5. All files appear as application/octet-stream on Cloudinary (encrypted blobs).

Deletion Notes

Cloudinary does not support unsigned deletes for security reasons. The deleteFile method in CloudinaryService is a stub that logs the deletion request. In production, this would require a signed backend endpoint with the API secret.

Section 9

Business Rules Summary

Storage Plans

Plan	Storage	Price	Status
Free	3 GB	Free	Enforced (quota check on upload)
Premium	20 GB	$19/month	UI only (no real IAP backend)
Professional	1 TB (Unlimited)	$49/month	UI only (no real IAP backend)

Security Rules

- Password policy: minimum 10 characters, must contain uppercase, lowercase, number, and special character
- Master key derived via Argon2id with 64MB memory cost
- Per-file encryption keys generated with CSPRNG
- Constant-time byte comparison to prevent timing attacks
- Auto-lock timeout: 2 minutes (configurable)

File Management Rules

- Trash retention: 30 days before auto-delete
- File comments: maximum 750 characters, plain text only
- Supported image previews: jpg, jpeg, png, gif, bmp, webp, heic, heif
- Supported video formats: mp4, mov, avi, mkv, webm
- Supported documents: pdf, txt, md, csv, json

Sharing Rules

- Vault files: personal only, no sharing allowed
- Papers wallet: cannot be shared externally
- Public links: optional expiry date and download limits
- User shares: X25519 key exchange for zero-knowledge sharing
- Share permissions: 'view' or 'download'
- Shares can be revoked at any time by the owner

Company Rules

- Company vault: removed (not implemented)
- Team sharing: file keys distributed to all team members

Section 10

Dependencies

Production Dependencies

Package	Version	Purpose
flutter	SDK	Core UI framework
cupertino_icons	^1.0.8	iOS-style icons
flutter_riverpod	^2.6.1	State management
riverpod_annotation	^2.6.1	Riverpod code generation annotations
go_router	^14.8.1	Declarative navigation & routing
firebase_core	^3.12.1	Firebase initialization
firebase_auth	^5.5.1	Authentication (email, Google)
cloud_firestore	^5.6.5	NoSQL database
firebase_app_check	^0.3.2+10	App integrity verification
cryptography	^2.7.0	XChaCha20-Poly1305, Argon2id, X25519
flutter_secure_storage	^9.2.4	Encrypted key storage
hive	^2.2.3	Local key-value storage
hive_flutter	^1.1.0	Hive Flutter integration
crypto	^3.0.6	Hash utilities
pointycastle	^4.0.0	Additional crypto primitives
google_mlkit_text_recognition	any	OCR for document scanning
flutter_animate	^4.5.2	Animation utilities
lottie	^3.3.1	Lottie animations
photo_view	^0.15.0	Zoomable image viewer
pdfx	^2.8.0	PDF rendering
flutter_pdfview	^1.3.2	PDF viewer widget
video_player	^2.9.2	Video playback
audioplayers	^6.1.0	Audio playback
google_fonts	^6.2.1	Google Fonts (Poppins)
shimmer	^3.0.0	Loading shimmer effects
file_picker	^8.1.7	File selection dialog
image_picker	^1.1.2	Camera/gallery image picker
camera	^0.11.0+2	Camera access for CamScanner
in_app_purchase	^3.2.0	In-app purchases (future)
connectivity_plus	^6.1.1	Network connectivity detection
path_provider	^2.1.5	File system paths
uuid	^4.5.1	UUID generation for IDs
intl	^0.20.2	Internationalization & date formatting
shared_preferences	^2.3.5	Simple key-value storage
image	^4.5.3	Image manipulation

mime	^2.0.0	MIME type detection
http	^1.2.0	HTTP client for Cloudinary
http_parser	^4.0.2	HTTP content-type parsing
collection	any	Collection utilities
equatable	^2.0.7	Value equality
freezed_annotation	^2.4.4	Freezed model annotations
json_annotation	^4.9.0	JSON serialization annotations
percent_indicator	^4.2.3	Circular/linear progress indicators
flutter_svg	^2.0.17	SVG rendering
share_plus	^10.1.4	Native share dialog
url_launcher	^6.3.1	Open URLs externally
permission_handler	^11.3.1	Runtime permission requests
workmanager	^0.9.0+3	Background task scheduling
flutter_local_notifications	^18.0.1	Local push notifications
csv	^6.0.0	CSV parsing
pdf	^3.11.2	PDF generation
google_sign_in	^6.2.1	Google OAuth sign-in
intl_phone_field	^3.2.0	Phone number input with country code

Dev Dependencies

Package	Version	Purpose
flutter_test	SDK	Testing framework
flutter_lints	^5.0.0	Lint rules
build_runner	^2.4.13	Code generation runner
riverpod_generator	^2.4.0	Riverpod provider generation
freezed	^2.5.2	Immutable model code generation
json_serializable	^6.8.0	JSON serialization code gen
hive_generator	^2.0.1	Hive type adapter generation
mockito	^5.4.4	Mocking for unit tests
integration_test	SDK	Integration testing framework

Section 11

Setup Instructions

Prerequisites

- Flutter SDK 3.x+ (Dart ^3.6.0)
- Android Studio or VS Code with Flutter extensions
- Android device or emulator (API 21+)
- Firebase project configured (storage-60b29)
- Node.js (for Firebase CLI)

Steps to Run

```
# 1. Clone the repository
git clone <repo-url>
cd Secure_Storage

# 2. Install Flutter dependencies
flutter pub get

# 3. Generate code (Freezed models, Riverpod, JSON serialization)
flutter pub run build_runner build --delete-conflicting-outputs

# 4. Connect an Android device or start an emulator
flutter devices

# 5. Run the app
flutter run
```

Firebase Setup

```
# Install Firebase CLI
npm install -g firebase-tools

# Login to Firebase
firebase login

# Configure FlutterFire (if not already done)
flutterfire configure --project=storage-60b29

# Deploy Firestore security rules
firebase deploy --only firestore:rules

# Deploy Firestore indexes
firebase deploy --only firestore:indexes
```

Firebase Console Configuration

- Enable Authentication providers: Email/Password + Google Sign-In
- Enable Cloud Firestore database
- Place google-services.json in android/app/
- Ensure firebase_options.dart is up to date in lib/

Cloudinary Configuration

- Cloud name: dqed2mebk
- Create unsigned upload preset: privault_uploads
- Allow auto resource type in upload preset settings

Environment Variables

Create a .env file in the project root (see .env.example for reference). The app reads Cloudinary and Firebase configuration from this file.